

OCJP JAVA 17 & 21 PROGRAMMER CERTIFICATION FUNDAMENTALS

OFFICIAL COURSE COMPANION

CHAPTER 19

Using Java I/O API

CHAPTER OVERVIEW

Java has a vast set of interfaces and classes for creating, reading, and modifying files. If you have never worked with files before, it might look intimidating at first but don't worry, I will start with a few simple...

EXAM OBJECTIVES COVERED

- ◆ Read and write console and file data using I/O Streams
- ◆ Serialize and de-serialize Java objects
- ◆ Create, traverse, read, and write Path objects and their properties using java.nio.file API

Chapter 19 Using Java I/O API

Exam Objectives

- Read and write console and file data using I/O Streams
- Serialize and de-serialize Java objects
- Create, traverse, read, and write Path objects and their properties using java.nio.file API

19.1 I/O Basics

Java has a vast set of interfaces and classes for creating, reading, and modifying files. If you have never worked with files before, it might look intimidating at first but don't worry, I will start with a few simple examples to give you a feel of the API and then get into the details required for the exam.

First, let's get our terminology straight.

19.1.1 I/O streams

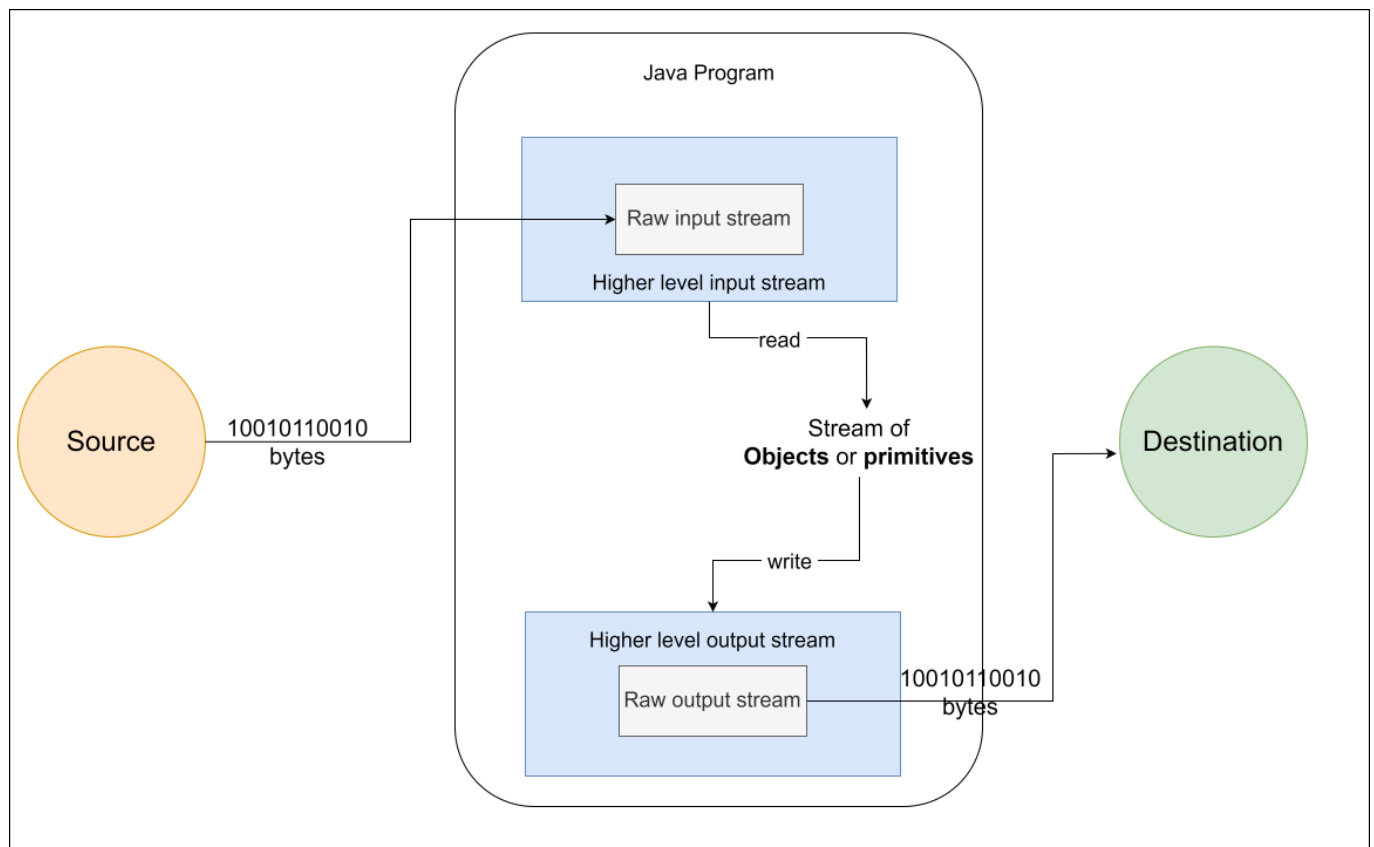
In the Java world, input and output of data to and from a Java program is done in the terms of bytes flowing into the program or out of the program. This flow of bytes is called "stream". So, a stream of bytes coming into a program is an "input" stream and a stream of bytes going out of a program is an "output" stream. A program reads the bytes coming in using an input stream and it writes the bytes that are meant to go out of the program using an output stream.

Note that usage of the word stream here has nothing to do with the Java Stream API. It is used only to evoke a mental picture of a stream of water because a stream of bytes

behaves quite similarly.

Since it is sometimes too tedious for a programmer to deal in raw bytes, the Java I/O library provides classes that wrap raw byte streams into streams of more meaningful data such as primitive types, text, and objects. Thus, for example, a programmer could use a wrapper stream to read characters instead of reading raw bytes. The wrapper stream implements the logic of converting raw bytes into character data and returns a stream of characters instead of bytes. One could again wrap the character stream into a stream that returns lines of text. The same applies for writing bytes. A programmer could directly write lines of text using an output stream that automatically converts lines of text into raw bytes, which can then be written to the destination.

Just like a stream of water, a stream of bytes has to start from somewhere (source) and end at somewhere (destination). The source or the destination of a data stream could be a file, a network socket, or even an in-memory array. There are classes in the Java standard library that let you work with commonly used sources and destinations. Thus, once you identify the source (or destination) and the format of the data, all you have to do is to chain and wrap existing classes to achieve your goal. This is illustrated by the following figure.



In the above image, observe that a Java program gets raw bytes from the source through a raw input stream and translates the raw bytes into data types needed by the program. Similarly, the program writes data types using a higher level stream, which converts them into bytes. The higher level stream then uses a raw output stream to write the bytes to the destination. The source or the destination do not care about how the bytes are interpreted or generated by the Java program.

Blocking vs Non-blocking I/O 🙌

For a long time, Java had only the classes in `java.io` package to perform I/O related tasks. However, these classes implement what is known as "**blocking**" I/O operations. They are called blocking because once you call a method to read or write data using these classes, that call will block until the operation is complete or until the stream has ended. For example, if you try to read data from a remote file or from a network socket, the thread on which the read method is being executed may be blocked for a long time and will not be available to perform other tasks. Although this approach is fine for many applications, it doesn't scale well. Imagine a server that is handling millions of connections from users and reads the data sent by them using

blocking I/O. If a thread is blocked on a read operation for even a few milliseconds, the server will grind to a halt very soon because there are only a limited number of threads. It is for this reason that older web servers used "thread pooling" to process requests.

To overcome this limitation, Java introduced a non-blocking I/O (aka NIO) library in Java 1.4 and updated it in Java 7 (aka NIO2). These classes are packaged in the **java.nio** package. Working with non-blocking I/O is substantially more complicated than working with blocking I/O but fortunately, this topic is not included in the exam. However, the nio library also includes the `java.nio.file` package which includes a few classes such as `Files` and `Paths` that are very helpful even while dealing with the blocking I/O and the functionality provided by this package is on the exam.

19.1.2 Reading and writing binary data 📌

The following program illustrates how you can read/write raw bytes directly from/to a file.

```
import java.io.*;

public class Test {

    public static void main(String[] args) throws IOException {
        InputStream fis = new FileInputStream("c:\\temp\\test1.png");
        int bite = fis.read();
        while (bite != -1) {
            byte data = (byte) bite;
            //do something with the byte just read
            System.out.print(data);
            bite = fis.read();
        }
    }
}
```

```

        fis.close(); //can be avoided by using try-with-resources

        try( OutputStream fos = new
FileOutputStream("c:\\temp\\test2.png") ){
            byte[] imageData = new byte[1000];
            for (byte b : imageData) {
                fos.write(b);
            }
        }
    }
}

```

The above program is a good starting point because even though it is fully functional, it has several shortcomings that necessitate the need for many of the important classes and interfaces of Java I/O API. Observe the following points about the above program:

1. The concept of raw input stream and raw output stream is represented by **abstract** classes `InputStream` and `OutputStream` respectively.
2. Since the source and destination of our input and output streams are files, we are using `FileInputStream` and `FileOutputStream` classes provided by the Java I/O library to read from and write to the files. They extend `InputStream` and `OutputStream` respectively.
3. The source of our input is a file located at `c:\\temp\\test1.png` and the destination of our output is another file located at `c:\\temp\\test2.png`. Note that these paths are Windows specific. If you are on Linux, you will have to use something like `/home/myname/temp/test1.png`. If you are thinking that the paths are totally making the program platform dependent, then you are right. They indeed are. There is a `java.io.File` class that helps us avoid platform specific path separators in the code. Furthermore, there is a **Path API** in `java.nio.file` package that helps us manipulate paths and interact with the file system in a platform independent manner.
4. Although the `read()` method reads a byte from the input stream, the return type of this method is `int`, which is why the variable `bite` has been declared to be of type `int`. Only the lower order 8 bits of the returned integer contain the actual byte read from the input stream. To get that value, we need to cast the

4. returned value to `byte` . Recall that casting an `int` value to `byte` removes 24 most significant bits (i.e. three higher order bytes) and leaves the 8 least significant bits of the given value intact.
5. We keep reading from the input stream until we get a `-1` . A `-1` indicates that the stream has ended and there is no more data available to read from the input stream.
6. Encountering exceptional situations while performing I/O is common. For example, the file that you are trying to open may not be present or the file that you are trying to write to may be marked read only by the OS. There are several specialized exception classes such as `FileNotFoundException` or `EOFException` defined in the standard library for this purpose, but all of them are rooted under the checked exception class `java.io.IOException` and so, the `throws IOException` clause in main satisfies all of them.
7. The writing part is unremarkable. It is just writing a thousand bytes one by one.
8. Finally, we use the `close()` method to close the input and output streams. Closing I/O streams using the `close()` method is important because it allows any system resources associated with the streams such as file locks to be reclaimed by the OS. Closing an output stream also flushes all data remaining in the output pipeline to the destination. Since `InputStream` and `OutputStream` implement `AutoCloseable`, the preferable approach is to let the try-with-resources statement close the streams automatically.

The Java API designers decided to return an `int` from `read()` instead of `byte` because an extra value was needed to signify the end of the stream. The total number of values returned by the method is 257, which includes 256 values (`00000000` to `11111111`) represented by a byte and an extra value of `-1` to denote the end of the stream. Since a `byte` cannot represent 257 values, a larger data type was needed. This could have been achieved using a `short` as well, but `int` was used to remain consistent with the C language.

Similarly, even though the `write(int )` method accepts an `int` parameter, only the lower 8 bits are actually written to the file. Although `byte` would have sufficed in this case, `int` was used

to be consistent with C.

Generally, reading data byte by byte is not too useful. For example, if you want to manipulate image data, you would need to load the complete image file in program memory and then process it. Similarly, writing to a file one byte at a time is quite inefficient. `InputStream` and `OutputStream` have a few methods that help you perform I/O operations in bulk. Here is an example that reads data from an `InputStream` and writes it to an `OutputStream` in chunks of 1024 bytes:

```
public static void transfer(InputStream is, OutputStream os) throws
IOException {
    byte[] chunk = new byte[1024];
    int bytesRead = -1;
    while( (bytesRead = is.read(chunk)) != -1 ){
        os.write(chunk, 0, bytesRead);
    };
    //not closing the streams here because this method hasn't
    opened them
}
```

In the above code, we pass a byte array of size `1024` to the `read` method. The `read` method populates the byte array with the bytes read from the input stream and returns the number of bytes that it actually read. It overwrites the byte array with the newly read bytes starting from index `0`.

To write the bytes to the output stream, we are using `OutputStream`'s `write(byte[] b, int offset, int len)` method, which writes `len` number of bytes from the given byte array starting from the index given by the `offset` parameter (which is `0` in our example). Note that the number of bytes read may be less than `1024` in the last iteration of the while loop and in that case, the byte array will still have old values read in the previous iteration at the index position indicated by `bytesRead` and beyond. Since we are only writing `bytesRead` number of values starting from index `0`, those old values will not be written to the output stream.

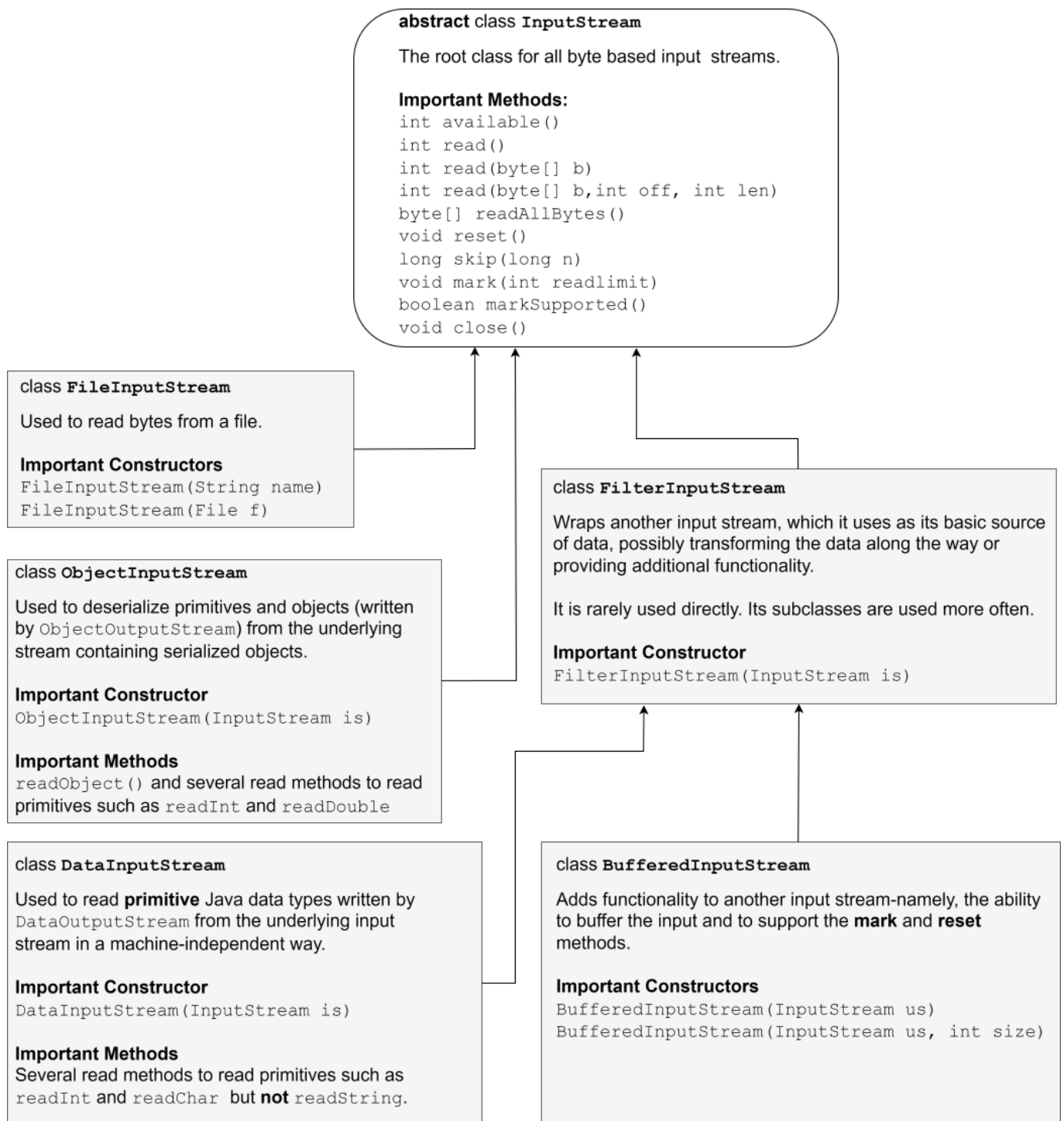
The size of `1024` is completely arbitrary. You can use any size that the program

environment permits. If you are reading bytes from a small file (may be about a few MBs), you may also use `InputStream`'s `available()` method to get the number of bytes that can be read immediately without blocking and then create a byte buffer of that size. Reading large chunks at once is not recommended because that may hog all of your program memory.

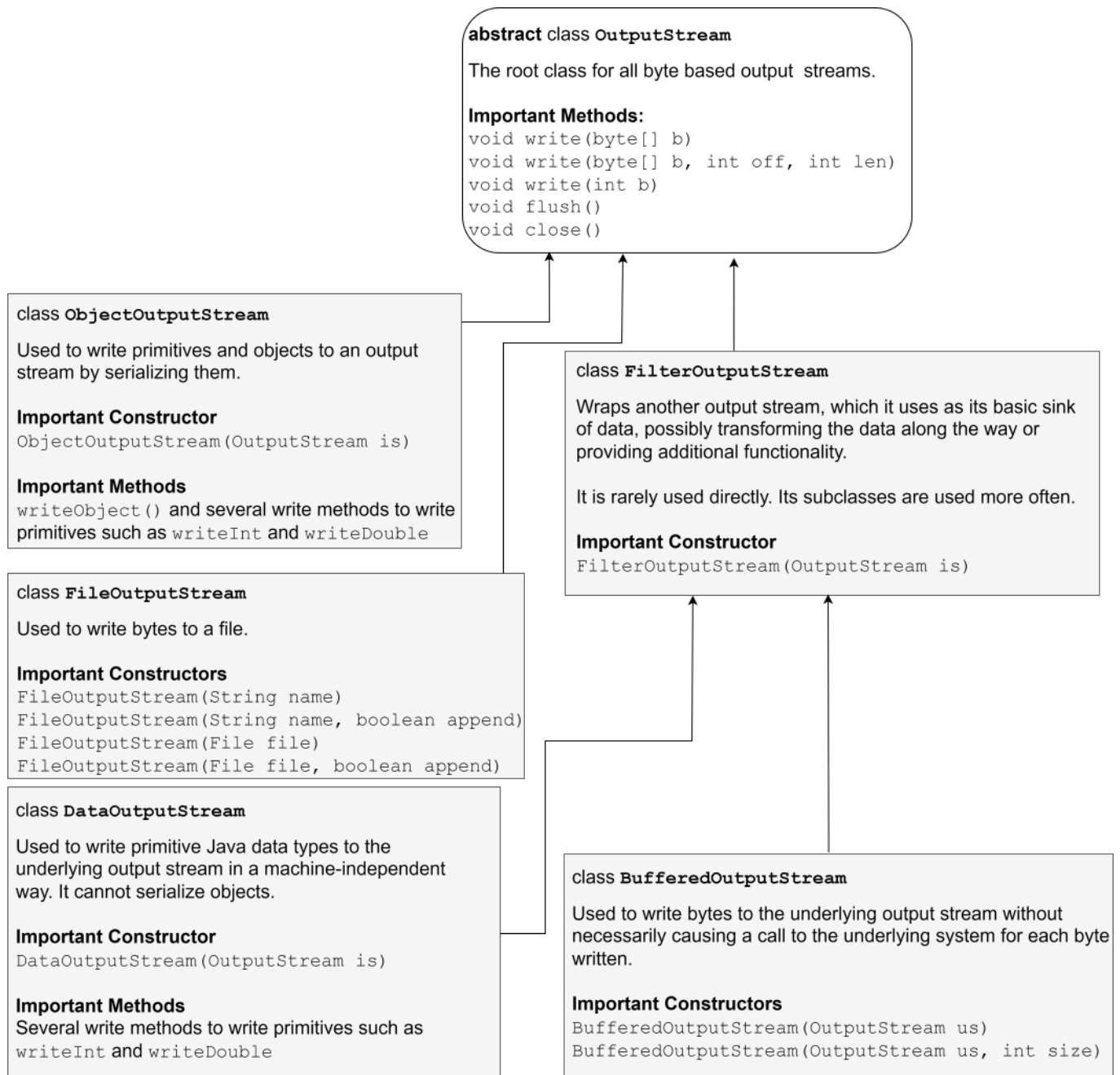
If this looks like too much work just to copy data from one stream to another then you are in luck! Java 9 has added a `transferTo(OutputStream )` method in `InputStream` that can achieve the above in just one line.

19.1.3 Summary of byte streams

The following image shows the important input streams that work with **bytes**.



The following image shows the important output streams that work with **bytes**.



The following are a few points worth noting about the classes shown above:

1. `InputStream` and `OutputStream` are the root classes for all input and output streams respectively. They are **abstract** and they contain most of the methods that we normally use while dealing with low level input and output.
2. Specialized classes such as `ObjectInputStream` (and its corresponding `ObjectOutputStream`) and `BufferedInputStream` (and its corresponding `BufferedOutputStream`) do not interact with the data source (or the data sink) such as a file directly. They take an existing input stream (or an existing

2. output stream) and build additional functionality on top of those streams. That is why such streams are called "higher-level" streams.
3. Higher level streams do not care about where the data actually comes from or goes to. It is the job of the lower level streams such as `FileInputStream` / `FileOutputStream` to deal with the actual data source or the sink.

Although not an official naming rule, a good trick to find out whether an I/O stream class constructor takes an input or an output stream as an argument is to see whether the name of the class reveals an actual data source/sink. If the name of an I/O stream class reveals an actual data source or sink, it is a lower level stream and it does **not** take an input or an output stream as an argument during instantiation. For example, `FileInputStream`, and `ByteArrayInputStream` reveal their data sources (a file and a byte array respectively) and are therefore, lower level streams. On the other hand, `ObjectInputStream`, `BufferedInputStream`, and `DataInputStream` reveal the type of the data that they deal with (Object, byte buffer, and primitive data, respectively) instead of the source or the sink of the data and are therefore, higher level streams. They do take an `InputStream` (or an `OutputStream`) as an argument during instantiation.

19.1.4 Reading and writing character data 🙌

You know that the actual data stored in a file system is a series of zeros and ones. It is our program that reads those bits and interprets them as something more meaningful such as characters. Thus, you can use raw `InputStream` and `OutputStream` classes to read raw bytes and interpret those bytes into characters on your own using some character encoding such as ASCII. However, converting bytes into characters is not always as trivial as interpreting a byte as an ASCII character. There are hundreds of languages and thousands of characters that exist and they cannot all be mapped directly to values of a byte. That is why modern character encodings use multiple

bytes to store a single character. Fortunately, Java provides `InputStreamReader` and `OutputStreamWriter` concrete classes that perform the conversion as per the given character encoding. An `InputStreamReader` reads raw bytes from an `InputStream` and converts them into characters on the fly. Similarly, an `OutputStreamWriter` converts a stream of characters to a stream of bytes on the fly and pipes the resulting stream of bytes to an `OutputStream` as shown in the following code:

```
FileOutputStream fos = new FileOutputStream("c:\\temp\\test.txt");
OutputStreamWriter osw = new OutputStreamWriter(fos, "UTF-8");
char[] charData = "hello".toCharArray();
for (char c : charData) {
    osw.write(c);
}
osw.close();
FileInputStream fis = new FileInputStream("c:\\temp\\test.txt");
InputStreamReader isr = new InputStreamReader(fis, "UTF-8");
int data = isr.read();
while (data != -1) {
    char letter = (char) data;
    System.out.print(letter);
    data = isr.read();
}
isr.close();
```

Observe the following points in the above code:

1. We are using `FileInputStream` and `FileOutputStream` to get raw input and output streams from and to a file. We are then using them to create `InputStreamReader` and `OutputStreamWriter` instances so that we can read/write characters instead of bytes from/to the file.
2. We are using `UTF-8` as the character encoding while writing and reading bytes

2. to and from the file. You do not need to know the details of any character encoding for the OCP exam but it is good to know about some of the commonly used character encodings such as `UTF-8`, `UTF-16`, and `ISO-8859-1`. `UTF-8` is the most popular of all and is also the recommended encoding for English only as well as for multilingual text because of its efficiency and its compatibility with ASCII.
3. Calling `close()` on `InputStreamReader` or `OutputStreamWriter` also closes the underlying `FileInputStream` and `FileOutputStream` instances.

Following the pattern of wrapping a lower level stream to create a higher level stream, we can read and write lines of text instead of single characters by wrapping an `InputStreamReader` into a `BufferedReader` and an `OutputStreamWriter` into a `BufferedWriter` as follows:

```
try( var fos = new FileOutputStream("c:\\temp\\test.txt");
    var osw = new OutputStreamWriter(fos, "ISO-8859-1");
    var bw = new BufferedWriter(osw);){
    String str = "hello";
    bw.write(str+"\r\n"+str+"\n\r");//writing three lines to the file
}

try(var fis = new FileInputStream("c:\\temp\\test.txt");
    var isr = new InputStreamReader(fis, "ISO-8859-1");
    var br = new BufferedReader(isr);){
    String str = null;
    while( (str = br.readLine()) != null){
        System.out.println(" | "+str+" | ");
    }
}
```

The above code prints:

```
|hello|
|hello|
```

```
||  
End of file
```

Observe the following points in the above code and output:

1. A total of **three** lines are read from the file. The third line is an empty string (it is not null). This is because a line is considered to be terminated by any one of a line feed (`\n`), a carriage return (`\r`), a carriage return followed immediately by a line feed (`\r\n`), or by reaching the end of the file. Note that `\n` is not a string containing two characters. It is just one character whose numeric value is 10. Similarly, `\r` is just one character whose numeric value is 13. When you write these characters in a file, they show up as line breaks in a text editor. To prevent the compiler from treating them as line breaks in our source code, we use escape sequences `\r` and `\n`.
In the above code, the first line is terminated by `\r\n`, the second line is terminated by `\n`, and the third line is terminated by `\r`. There is no fourth line in the file.
2. We keep on reading the lines until the `BufferedReader` returns `null`.
3. We are using the **try-with resources** statement with multiple resources and **var declarations**. This is typical while using I/O streams.

Java also provides concrete classes named `FileReader` and `FileWriter` that extend `InputStreamReader` and `OutputStreamWriter` respectively for reading character data. Here is a sample program that shows how to use these classes:

```
try(var fw = new FileWriter("c:\\temp\\test.txt", Charset.forName("UTF-8"))) ){  
    char[] charData = "hello".toCharArray();  
    for (char c : charData) {  
        fw.write(c);  
    }  
}  
try(var fr = new FileReader("c:\\temp\\test.txt", Charset.forName("UTF-8"))) ){
```

```
int data = fr.read();
while(data ≠ -1) {
    char letter = (char) data; //cast data to char
    //do something with the char just read
    System.out.print(letter);
    data = fr.read();
}
}
```

The above program is similar to the previous ones except that it uses an instance of the `java.nio.charset.Charset` class instead of a `String` to specify the character encoding. `Charset` is a utility class that contains the mapping between raw byte data ("code units", to be precise but you can ignore that for the exam) and the characters.

Character data **always** requires an encoding and decoding mechanism to convert characters into raw bytes and vice-versa. The rules for this interpretation are defined by a character encoding. Although it is usually possible to read and write character data without specifying a character encoding explicitly, it is a bad idea to do so because not specifying a character encoding explicitly just means that some default character encoding is applied by the underlying API implicitly. Any change in the default encoding may render the existing character data completely incomprehensible. A change in the default character encoding could happen because of migration or upgradation of the underlying data store such as the file system or the database.

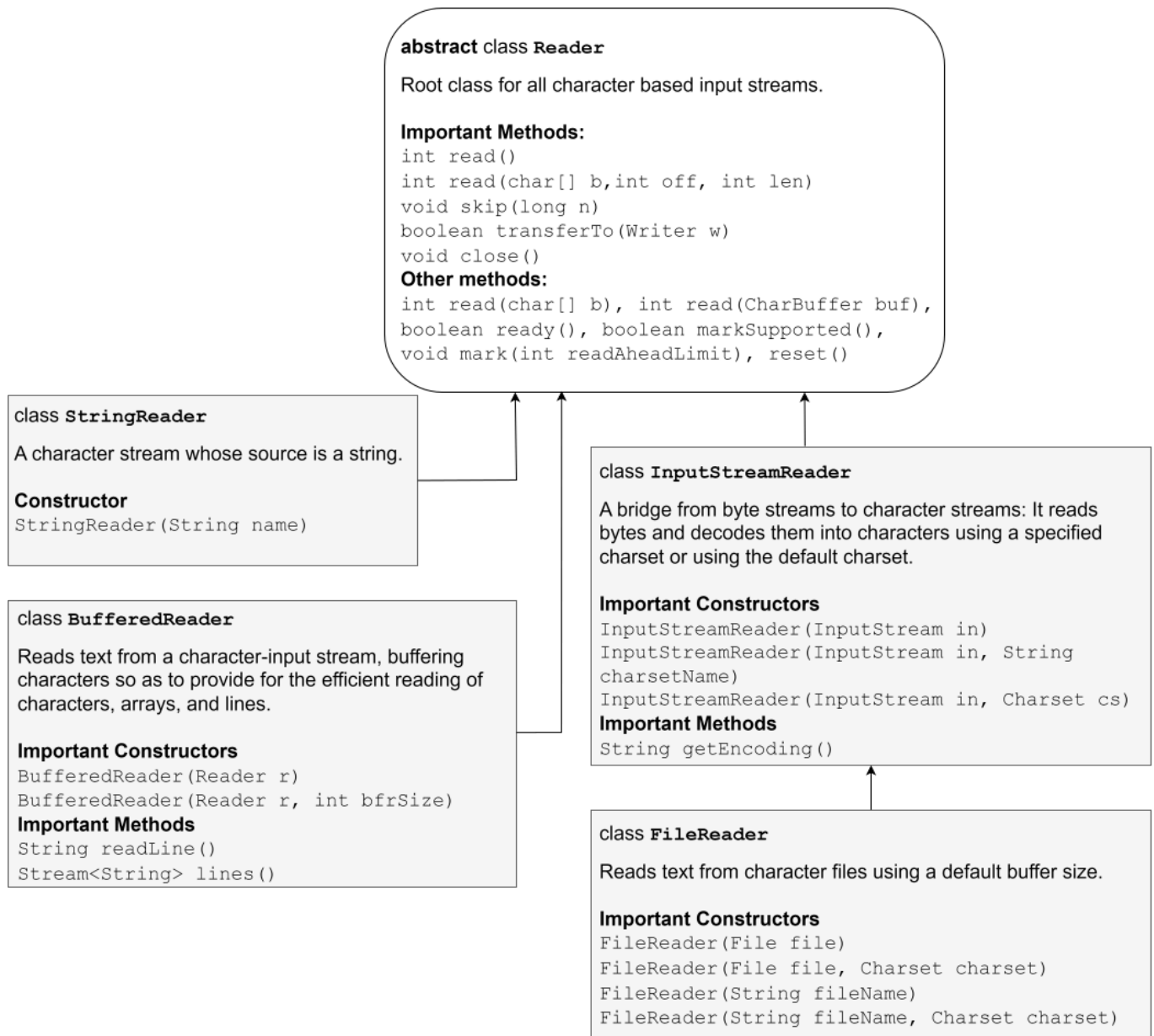
Prior to Java 11, `FileReader` and `FileWriter` did not take a `Charset` as an argument during instantiation. They used the default encoding as determined from the environment in which the program is run. This behavior leaves the programmer unaware of the conversion mechanism and renders the data

potentially non-portable. That is why the preferred way to read and write character data prior to Java 11 was to use `InputStreamReader` and `OutputStreamWriter` classes, which take the name of the character encoding as an argument.

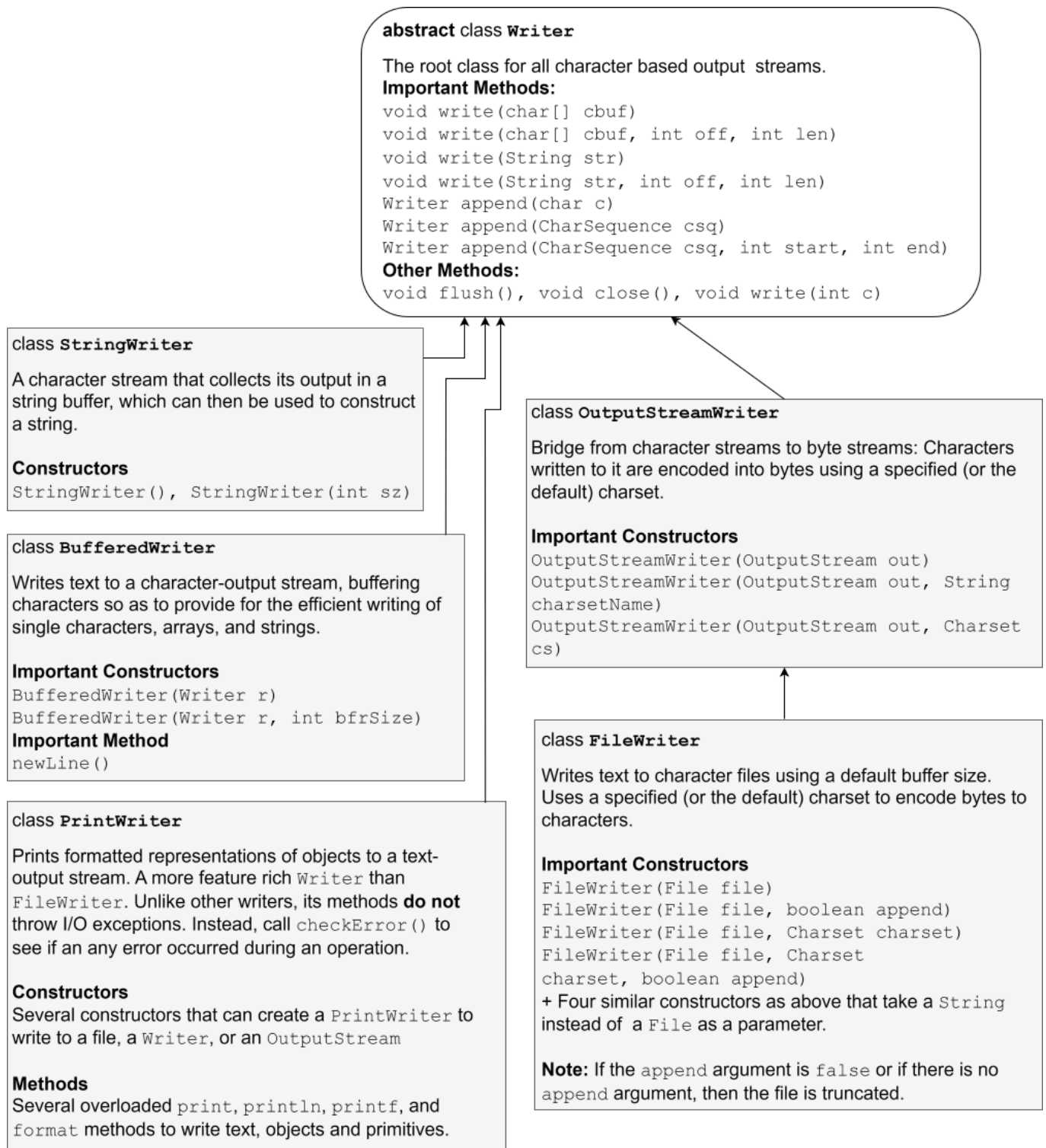
In Java 18, `FileReader` and `FileWriter` and several other classes were updated to use UTF-8 as the default character encoding instead of relying on the default character encoding of the environment. Although this change does make the code more portable, it is still better to specify the character encoding explicitly.

19.1.5 Summary of character streams

The following image shows the important input streams that work with **characters**.



The following image shows the important output streams that work with **characters**.



The following are a few points worth noting about the classes shown above:

1. `Reader` and `Writer` are the root classes for all character based input and output streams respectively. They are **abstract** and they contain most of the methods that we normally use while dealing with low level character based input

1. and output.
2. `InputStreamReader` and `OutputStreamWriter` classes act as a bridge between a byte stream and a character stream. They convert a byte stream to a character stream (and vice-versa) by using the specified (or the default) character encoding.
3. Some classes such as `BufferedReader` and `BufferedWriter` require an instance of a bridge class (i.e. `InputStreamReader` or `OutputStreamWriter`) as a constructor argument, while others such as `FileReader`, `FileWriter`, and `PrintWriter` can be created even without such bridge classes. Such classes create an intermediate `InputStreamReader` or `OutputStreamWriter` object internally to perform the conversion between bytes and characters.
4. The above summary does not include `PrintStream`. Although it is used very often to print just about everything to the console (using `System.out`), `PrintStream` is not a `Writer`. I will talk about it separately.

19.1.6 Advanced I/O stream operations

Rewinding a stream

Generally, data from an input stream can only be read once. You cannot "rewind" a stream and reread the same data again. However, some input streams that use internal data buffers may be able to provide this functionality. This functionality comes in handy while implementing parsers that need to rewind to a previous location if they encounter a specific control character. For example, the following codes reads a text file and identifies words enclosed within `<` and `>`. The logic is as follows. We "mark" the stream when we encounter a `<` and start counting the characters. We count till we encounter a `>`. As soon as we encounter a `>`, we reset the stream back to the marked location and read the number of characters indicated by the count in a character buffer.

```

try(BufferedReader bfr = new BufferedReader(new
FileReader("test.txt"))){
    if(bfr.markSupported()) { //check whether the stream supports
mark/reset
        int letter = -1;
        int noOfChars = 0;
        String tag = "";
        while ((letter = bfr.read()) != -1) {
            noOfChars++;
            if (((char) letter) == '<') {
                noOfChars = 0; //start counting chars till we hit '>'
                bfr.mark(256); //mark this location (just after '<')
            }
            if (letter == '>') {
                bfr.reset(); //rewind to the marked point
                char[] buf = new char[noOfChars - 1];
                bfr.read(buf);
                System.out.println(new String(buf)+" ");
                bfr.skip(1); //skip the '>' character
            }
        }
    }
}

```

If the `test.txt` file contains `<html> </html>`, the above code will print `html /html`.

Here are a few points worth nothing about the above code:

1. The `markSupported` method tells you whether the underlying stream supports mark and reset.
2. The `mark` method takes an `int` parameter named `markAheadLimit`. It sets the maximum number of characters you can read safely after which the stream may invalidate the mark. In the above code, we have passed 256, which means, if we read more than 256 characters after setting the mark, we may not be able to

2. reset the stream back to that mark. The stream will throw an `IOException` if we reset the stream after the stream has already invalidated the mark.
3. The `skip` method skips the stream forward by the specified number of characters.

Note that the above example uses a `Reader` (which means, it is a character stream) but this functionality works the same way for an `InputStream` (which means, it is a byte stream) as well. The only difference is that in case of a byte stream, the arguments to `mark` and `skip` methods are interpreted as the number of bytes (instead of number of characters).

19.1.7 Quiz 🙋

Q1. Given that `test1.txt` contains the following two lines of text:

```
hello
world
```

(Assume that the first line is terminated by `\r\n`. What will be the contents of `test2.txt` after the following code is run?

```
char[] buff = new char[5];
try(var fr = new FileReader("c:\\temp\\test1.txt");
    var fw = new PrintWriter("c:\\temp\\test2.txt")){
    int count = 0;
    while( (count=fr.read(buff)) != -1) {
        fw.write(buff);
    }
}
```

Select **1** correct option.

A.

```
hello
```

world

B.

hello

worldwor

C.

hello

world

w

D.

hello

world

wor

Correct answer is B.

There are a total of 12 characters in `test1.txt` (don't forget the two characters used for terminating the first line). In the first iteration of the while loop, the read method fills `buff` with the first 5 chars, i.e., `hello`. Observe that we have not specified the offset and the length arguments to the `write` method. So, it writes all of the characters in `buff` to `test2.txt`. Thus, `hello` is written to the file. In the second iteration, the read method reads the next 5 chars, i.e., `\r\nwor`, which are then written to the file. Now there are only 2 more characters left to be read from the file. Thus, in the third iteration, the read method reads only those characters and puts them in the first two locations of `buff`. The remaining three locations in `buff` are unaffected and still contain the characters read in the previous iteration, i.e., `wor`. So, `buff` now contains `ldwor`. Thus, the write method writes `ldwor` to the file. The fourth iteration does not happen because the read method returns `-1`.

Had we used `fw.write(buff, 0, count)` instead of `fw.write(buff)`, the program would have copied the contents correctly.

19.1.8 Understanding file names and paths

I am sure you have been managing files in your computer using the File Explorer

since a long time and you don't need an elementary level tutorial on how to organize files and folders (aka directories). Even so, let me talk about a few terms that will be used extensively while learning about the Path API in the next section.

Files, Directories, and Folders

When you look at your file system through the file explorer, you see folders and files. Folders contain files and/or other folders, while files contain the data. Observe that a folder may contain multiple files and folders but a file or a folder can be contained exactly within one folder. This is what makes it a hierarchical file system. If you use the command line (or the shell), you can observe the same hierarchical structure in terms of files and directories using the `dir` command on Windows (or `ls` on Linux). At the top of a hierarchical file system is the root of that file system. In Windows, a root is denoted by a drive letter such as `c:` and in Linux, it is denoted by just a forward slash (`/`).

In the Java world, however, there is a single abstraction called "file" for files as well as directories. A file could be a regular file or it could be a directory (aka a folder). You can check what it is by inquiring its "attributes". So, if a file is a directory, you can invoke directory related operations on it such as retrieve the names of the files that it contains and if it is a regular file, you can read/write data from/to that file using various type of streams that you saw earlier.

File name

The name of a file is just a string that identifies the file within its parent directory. Conventionally, names for regular files end with a dot and a multi lettered extension such as `.txt` or `.java` while names of directories do not.

Pathname and Abstract pathname

An operating system uses a pathname to identify a file, while Java uses an abstract pathname for the same. The JVM converts an abstract pathname to the real pathname and vice-versa as and when required. The conversion logic depends on the operating system.

An abstract pathname is a series of zero or more file names separated by a system specific separator character. On Windows the separator character is back slash (`\`),

while on Linux and MacOS, it is forward slash (/). It is not necessary for an abstract pathname to refer to an actual file in the file system, but if you want to be able to convert it to a real pathname, every component except the last one of an abstract pathname must correspond to a directory. This makes sense because a file cannot contain other files, only a directory can.

Ideally, you should use the platform specific separator character while writing pathnames in your code (if you want to hardcode them in your code at all) but on Windows, the JVM automatically converts forward slashes to back slashes and so, if you use forward slashes in your code, the path will be acceptable on Windows as well as on Unix like machines. You will learn better ways to work with paths in a platform independent manner soon.

Absolute path and relative path

To access a file in a hierarchical file system, we need to know not just the name of the file but also the names of its containing folders, all the way up to the root. So, if a pathname includes all the names starting from the root down to the file that you want to access, it is an absolute path. For example, `c:\a\b\c\test.txt` is an absolute path because we can start from the root `c:\` and step inside the folder named `a`, then in `b`, and `c` to reach the `test.txt` file. Similarly, on Linux, since the root is denoted by `/`, `/a/b/c/test.txt` would be an absolute path.

A relative path, on the other hand, is not sufficient in itself to identify a file in a file system. It can only identify a file when used in relation to another path whose location is already known. For example, `b\c\test.txt` is not sufficient in itself to locate the `test.txt` file. However, if we are told that this path is relative to `c:\a`, then we can figure out its exact location. Thus, any path that does not start with a root is a relative path.

Current directory

The current directory is the path from where a program is run. For example, if you run your Java program from `c:\temp`, then `c:\temp` will be the current directory irrespective of where the class file being executed is located. When you execute a Jar file directly by double clicking on it, Windows sets the current directory as directory in

which the jar file is located. On older Windows and Java versions, the current directory is `c:\windows\system32`.

Knowing the current directory is important because all relative paths are resolved against the current directory by default. So, if your program tries to open a file whose relative path is `a\test.txt` file and if the current directory is `c:\temp`, then the JVM will expect the file to be located at `c:\temp\a\test.txt`.

Dot (.) and dot dot (..)

You need to be aware that all directories in Unix and DOS based files systems automatically get two children named `.` and `..`, where `.` refers to the same directory as the one in which it exists and `..` refers to its parent directory. For example, if you execute "`dir .`" command (without the double quotes) in `c:\temp`, you will see a listing of the files present in `c:\temp` and if you run "`dir ..`" command, you will see a listing of files present in the parent of `temp`, i.e., `c:\`. You will observe the same on a Linux machine as well. The reason and purpose of their existence has a lot to do with how a file system is managed by an OS, but you are not required to learn those details for the exam. You just need to know how they affect a pathname. A dot means, stay in the same directory, while a dot dot means, go a level up. So, for example, `c:\temp\a\..` means, starting from `c:`, go to `temp`, then to `a`, and then back to the parent of `a`, which means, you are back to `c:\temp`. Similarly, if do `cd ..` while in `c:\temp` on the command prompt, you will be in `c:`.

Canonical path

A canonical path is an absolute path that doesn't contain any redundant path fragments. For example, `c:\temp\a\..\test.txt` is an absolute path but if you follow this path, you will go from `c:` to `temp`, then from `temp` to `a`, then back to `temp`, and finally to `test.txt`. The presence of `..` cancels out the `a` that appears before it. Thus, `a\..` forms a kind of loop in the path. A canonical path irons out all such loops and gives you a straight path from the root to the destination.

Some operating systems allow you to create symbolic links to other files in the file system. A canonical path resolves such links as well, meaning, it shows the real path instead of the symbolic link.

19.1.9 Using the java.io.File class 🙌

You have already seen most of the important classes of the `java.io` package. The only other class that you need to know about from this package is the `File` class. It is an **immutable** class whose objects represent a file or a directory path name. The purpose of this class is not to operate on the data inside the file but to work with the file itself in a platform independent manner.

Creation, renaming and deletion

The following code illustrates how to **create**, **rename**, and **delete** the actual file in the file system using methods of the `File` class:

```
File f1 = new File("c:\\temp\\test1.txt");
boolean successful = f1.createNewFile();
System.out.println(successful);//prints true if test1.txt did not
preexist.
File f2 = new File("c:\\temp\\test2.txt");
System.out.println(f1.renameTo(f2));//prints true if test1.txt was
renamed to test2.txt.
System.out.println(f2.delete());//prints true if test2.txt was deleted.
```

Observe that a `File` object representing any file can be created irrespective of whether the file actually exists or not in the file system. The `createNewFile` method returns `true` if it was able to create an empty file in the file system. It could fail (and return `false`) if the file already exists or if there is a security restriction. The `renameTo`, `makeDir`, `makeDirs`, and `delete` methods work similarly.

Notably, the `File` class does **not** have methods to **copy** and **move** a file. The `renameTo` method can be used to move the file but it may fail if the destination is on a different file system or if the destination already exists. The

`java.nio.file.Files` class provides a more comprehensive way to do these operations.

Listing a directory

You may use a `File` object to list the contents of a directory if the `File` object corresponds to a directory in the file system. For example, the following code prints names of all the files present in `c:\\temp` directory:

```
File f = new File("c:\\temp");
if(f.isDirectory()){
    String[] fileNames = f.list();
    for(String fn : fileNames){
        System.out.println(fn);
    }
}
```

Using platform independent name separators

In the previous code examples, I used Windows specific paths to files such as `"c:\\temp\\test1.txt"` and `"c:\\temp"`. These path strings won't work on a Linux machine because Linux uses forward slash (`/`) as the separator character. Furthermore, the root of a file system in Linux is just `/`. Linux doesn't use drive letters such as `c:`. The `File` class has two public static fields for this purpose:

1. `public static char separatorChar`: It is the system-dependent default name-separator character. On Windows it is back slash (`'\\'`) and on Linux, it is forward slash (`'/'`).
2. `public static String separator`: A `String` version of `separatorChar` for convenience. It contains the same character as `separatorChar`.

So, if I write the path string `"c:\\temp\\test.txt"` as `File.separator + "temp" + File.separator + "test.txt"`, it will work on Windows as well as on Linux because it will be translated to `\\temp\\test.txt` if the program is run on a Windows machine and to `/temp/test.txt` if it is run on a Linux machine. Yes, the

translation on Windows is missing the drive letter `c:`, but unless you want to target a file on a different drive than the one from which your program is executed, a drive letter is not required for specifying absolute paths. Any path that starts with the separator character is assumed to be starting from the root of the file system (on all OSs). Thus, on a Windows machine, if your path starts with a `\` and your current directory is on the `C` drive, then `\temp\test.txt` will refer to `c:\temp\test.txt`.

The `File` class has two more public static fields for separating paths in a platform independent manner. These are:

1. `public static char pathSeparatorChar`: It is the system-dependent path-separator character. On windows, it is `;` and on Linux, it is `:`.
2. `public static String pathSeparator`: A String version of `pathSeparatorChar` for convenience. It contains the same character as `pathSeparatorChar`.

Although completely beyond the scope of the exam, it is good to know that these fields come in handy when you want to dynamically execute a command that expects a list of paths as an argument.

Getting path information

The `File` class has methods such as `getAbsolutePath` and `getCanonicalPath` to get the absolute path and the canonical path respectively of the file. They can be used as follows:

```
File f = new File(".");
String ap = f.getAbsolutePath();
String cp = f.getCanonicalPath();
System.out.println(ap+" "+cp);
```

The above code uses a relative path while creating the `File` object (`.` is relative because it does not start with the root). This path is relative to the current directory. Thus, effectively, the above code shows you the current directory. If you run the above code from `c:\temp`, it will print `C:\temp\.` `C:\temp`. Observe that the canonical path does not contain a dot at the end because the dot is redundant.

Getting file attributes

The `File` class lets you enquire about its properties of the file through methods such as `isDirectory()`, `isFile()`, `isHidden()`, and `lastModified()`.

The `File` class has been in existence since Java 1.0 and is usually sufficient for routine things. However, since the exam does not focus on the `File` class much, I am not going into the details of all its methods. I suggest you to browse through the JavaDoc for the `File` class and write a few test programs to see how they work.

19.2 The `java.nio.file` package

The `java.nio.file` package provides a more comprehensive and platform independent access to the file system than the `java.io.File` class. It does so, by formally abstracting the concepts of file system, path, and file attributes into a large set of interfaces and classes. Some of the things that you can do using this package that you cannot do using the `File` class are - open files in ten different read/write modes, copy files and directories, refer to files in a different file systems, watch directories for changes at run time, get finer control over symbolic links, recursively traversing directories, synchronize writes to the file system, and managing file attributes. Yes, that's a lot of things and there are more! But do not worry. You are not expected to know all of these things. Usually developers learn how to perform these tasks only when they actually need them on a project.

The OCP exam also does not expect you to know about most of the above mentioned features and focuses only on one feature - working with paths and their properties.

19.2.1 Important classes and interfaces of the `java.nio.file` package

The following is a list of interfaces and classes from the java.nio.file package that you need to be aware of for the OCP exam.

Interface Important Methods

Instance methods:

```
getName(int index), getParent(), getRoot(), subpath(int  
beginIndex, int endIndex), toAbsolutePath(),  
toRealPath(LinkOption... options), toFile()  
normalize(), relativize(Path other), resolve(Path other)
```

Static method:

```
of(String first, String... more)
```

OpenOptions
CopyOptions

Class

FileSystems

Contains only static factory methods for file systems.

FileSystem

Provides an interface to a file system and is the factory for objects to access files and other objects in the file system.

Paths (Not recommended for use anymore)

Contains only static methods that return a Path by converting a path string or a URI.

Important Methods

Static methods:

```
getDefault(), getFileSystem(URI uri)
```

Instance methods:

```
getPath(String first, String... more),  
getRootDirectories(), getSeparator()
```

Static methods:

```
get(String first, String...  
more), get(URI uri)
```

Methods to copy, create, delete, and move files as well as directories.

Methods to list a directory - `Stream<Path> list(Path dir)`

Methods to read character data such as `Stream<String> lines(Path path)`, `List<String> readAllLines(Path path)`, `String readString(Path p)`

Methods to walk a directory tree - `Stream<Path> walk(Path start, int maxDepth, FileVisitOption... options)` and `Stream<Path> walk(Path start, FileVisitOption... options)`

Methods to read binary data such as `byte[] readAllBytes(Path p)`

Files

Contains only static methods that operate on files and directories.

Methods to get stream based readers and writers for binary and character data such as:

`InputStream newInputStream(Path path, OpenOption... options)`, `OutputStream newOutputStream(Path path, OpenOption... options)`, `newBufferedReader(Path path)`, and `newBufferedWriter(Path path, OpenOption... options)`

Methods to write character and binary data to a file such as `write(Path path, Iterable lines, OpenOption... options)`, `writeString(Path path, CharSequence csq, OpenOption... options)`, `write(Path path, byte[] bytes, OpenOption... options)`

Several `isXXX()` methods to enquire file attributes such as `isRegularFile`, `isReadable`, `isWritable`, `isHidden`, and `isSymbolicLink`.

enum	StandardOpenOption	Defines 10 modes:	READ, WRITE, APPEND,
implements	OpenOption		TRUNCATE_EXISTING, CREATE, CREATE_NEW,
			DELETE_ON_CLOSE, DSYNC, SYNC, SPARSE
enum	StandardCopyOption	Defines 3 modes:	ATOMIC_MOVE, COPY_ATTRIBUTES,
implements	CopyOption		REPLACE_EXISTING
enum	LinkOption	implements	Defines 1 mode: NOFOLLOW_LINKS
	OpenOption, CopyOption		

19.2.2 Using the Path interface 📌

You have already seen file names and paths in the previous section. They were all just strings and they used platform specific separators. The `java.nio.file` package formally abstracts the concept of a path using the `java.nio.file.Path` interface. Other classes of this package use `Path` objects instead of strings whenever they need to refer to a file.

There is no publicly accessible class that implements the `Path` interface, which means, you cannot instantiate `Path` objects directly. You need to get `Path` objects using either an existing `Path` object or by using the static `of( ... )` methods of the `Path` interface.

The following example shows the usage of a few commonly used methods of the `Path` interface. The output is (shown in comments) assumes that it is run from `C:\temp`:

```
Path currentDirPath = Path.of("./files");
System.out.println(currentDirPath.getFileName()); // files
System.out.println(currentDirPath.subpath(1, 2)); // files
System.out.println(currentDirPath.getRoot()); // null
Path absPath = currentDirPath.toAbsolutePath();
System.out.println(absPath); // C:\temp\.\files
Path normalizedCurrentDirPath = absPath.normalize();
System.out.println(normalizedCurrentDirPath); // C:\temp\files
```

```
System.out.print(normalizedCurrentDirPath.toRealPath()); //  
java.nio.file.NoSuchFileException
```

The following are a few points worth noting about the above example:

1. The `of` method provides a way to convert a `String` into a `Path` object. The strings used to create `Path` objects follow the same conventions and syntax as the ones you saw in the previous section. Thus, `./files` is a relative path because it doesn't start with a root component. It will be interpreted as being relative to the current directory, which is `C:\temp` in this case.
2. The `getFileName` method returns the farthest (i.e. the last) element of the path, which is `files` in this case.
3. The `subpath` method returns a relative path that is a subsequence of the name elements of this path. The returned `Path` object has the name elements that begin at `beginIndex` and extend to the element at index `endIndex-1`. Note that it returns a relative path, which means, the returned path will never start with the root.
4. The `getRoot` method returns the root if it is invoked on an absolute path. Since `./files` is not an absolute path, `getRoot` returns `null` here.
5. The `toAbsolutePath()` method returns the full path of the given relative path starting from the default root. In this case, the root is `C:\` and so, it returns `C:\temp\.\files`. Observe that this path contains a `.`, which is redundant. This shows that the `toAbsolutePath` method does not return the canonical version of the path. To get that, we need to call `normalize()`. The `normalize()` method is practically the same as the `getCanonicalPath` method of the `java.io.File` class (except the fact that `normalize()` returns a `Path` object instead of a `String`).
6. The `toRealPath` method returns the canonical path for the given path and also checks if the file (or the directory) referred to by the path actually exists. If the file does not exist, it throws an `IOException`. Thus, in this example, if the `files` directory exists, it will return `C:\temp\files` and if it does not, it will throw an `IOException`. (In reality, a `java.nio.file.NoSuchFileException` is thrown but since the JavaDoc says it throws `IOException`, we will go with `IOException`. `NoSuchFileException` is an `IOException`.) In fact, this is one of the only

6. two methods of the `Path` interface that can throw an `IOException`. The other method is `register`, but it is not in scope for this exam.

Not important for the exam, but it is good to know that a well formed paths do not end with a trailing separator character (unless it is referring to the root such as `C:\` or `/`). So, even if you try to create a `Path` with a trailing slash, it will be ignored. For example: `System.out.print(Path.of("/a/b/"));` will print `/a/b` and not `/a/b/`.

Using `resolve` and `resolveSibling` 🙌

Absolute paths on different environments in which an application runs, seldom match. For example, let's say your application needs to read a properties file named `values.properties`. On your Windows machine, you may be running this application under `C:\dev\projects\fintech` and this file may be located at `C:\dev\projects\fintech\props\values.properties`. But in production, the application may be running under `/home/ec2-user/prod/fintech` and the actual location of this file may be `/home/ec2-user/prod/fintech/props/values.properties`. Obviously, if you hardcode the absolute path to this file in your code based on your development environment, the application will not be able to find this file in production. Therefore, it is advisable to use relative paths instead of absolute paths in Java code. The base path is provided to the application through a command line argument and the application "resolves" the relative paths (which are same on all environments) relative to the base path whenever it needs to locate the file in the file system. The following example shows how it is done:

```
import java.io.*;
import java.nio.file.*;
public class TestClass{
    public static void main(String[] args) throws IOException{
        Path basePath = Path.of(System.getProperty("basepath"));
```

```

Path propFilePath = basePath.resolve("props/values.properties");
System.out.println(propFilePath);
Path dbPath = propFilePath.resolveSibling("dbconnection.properties");
System.out.println(dbPath);
}
}

```

You can create and run the above `TestClass.java` file from any directory but run it with the following command:

```
java -Dbasepath=c:\temp TestClass
```

The `-D` option passes a system property named `basepath` with a value of `c:\temp` to the program, which the program can read by calling `System.getProperty("basepath")`.

If the argument to the `resolve` method is a relative path, the `resolve` method assumes that the given path is relative to the path on which this method is called. Therefore, it just joins the two paths to create the actual path to the file. In this case, since `basePath` is `c:\temp`, `basePath.resolve("props\values.properties");` returns `c:\temp\props\values.properties`. If the argument is an absolute path, there is nothing to resolve and it returns the same path as the argument.

The `resolveSibling` method works similarly. The only difference is, it assumes that the path given as an argument is supposed to exist at the same level as the path on which this method is invoked. Therefore, `propFilePath.resolveSibling("dbconnection.properties")` returns `c:\temp\props\dbconnection.properties`. Had we called `propFilePath.resolve("dbconnection.properties")`, it would have returned `c:\temp\props\values.properties\dbconnection.properties`.

Finding relative path using `relativize` 🙌

The `relativize` method is the inverse of `resolve`. It finds a path to the given file relative to the path on which it is invoked. In other words, it tells you the path that you need to take to reach the given path from the path on which it is called. Continuing the previous example, given that `basePath` is `c:\temp` and that `propFilePath` is `c:\temp\props\values.properties`, `basePath.relativize(propFilePath)` will return `props/values.properties`.

Although the exam does not have tricky questions on this topic, the following paragraph from the JavaDoc description of this method is worth reading:

"A relative path cannot be constructed if only one of the paths have a root component. Where both paths have a root component then it is implementation dependent if a relative path can be constructed. If this path and the given path are equal then an empty path is returned. For any two normalized paths `p` and `q`, where `q` does not have a root component, `p.relativize(p.resolve(q)).equals(q)`."

Ideally, the `resolve` and `relativize` methods make sense when they are invoked on paths representing a directory (the argument can be a file or a directory). Since a `Path` object makes no distinction between a file and a directory, you can invoke these methods even if the `Path` you invoke them on represents a file. However, computing the result of these methods mentally becomes a bit confusing when the `Path` represents a file. For example:

```
Path file1Path = Path.of("c:\\temp\\props\\values.properties");
Path file2Path = Path.of("c:\\temp\\props\\log.txt");
Path relativePath = file1Path.relativize(file2Path); //returns
.. /log.txt
```

In the above code, you are trying to go from `file1Path` to

`file2Path` and `relativize` returns `../log.txt`. If you actually try to use the result as a series of steps that you need to take to reach `log.txt` from `values.properties` on the command line, you will find that the steps are incorrect. The `..` expects you to step up one directory and then find `log.txt`. However, `values.properties` is actually a file and not a directory. You cannot be inside `values.properties` to begin with. You can only be in the `props` directory and if you go one level up from the `props` directory, you will be in the `temp` directory, which does not have `log.txt`.

Therefore, while working out the result you need to assume that the path is a directory and the steps are given assuming you are in that directory. Thus, if you are, hypothetically, in the `values.properties` directory, you need to go up one directory to `props`, and then you will find `log.txt` there.

File and Path interoperability 🙌

If you want to work exclusively with the `Path` interface, you can. You will not need to use the old `java.io.File` class for anything. However, old code and third party libraries that your code depends on may still be using the `File` class. Both - the `File` class and the `Path` interface - have methods that allow you to switch between the two easily. Specifically, `File` has a `toPath` method that returns a `Path` object representing the same file and `Path` has a `toFile` method that returns the corresponding `File` object.

19.2.3 Using the Files class 🙌

While going through the methods in the `java.nio.file.Files` class, you will notice that it is a convenience class that offers a ton of methods and they let you do pretty much everything that you may want to do with files. In fact, all of its methods are static and they delegate to the associated file system provider to perform file

operations.

Although the exam does not focus directly on this class, a few of its methods appear frequently in the questions. I will go over only such methods in this section. Just remember that this class uses `Path` instead of a `String` to refer to a file.

Getting I/O streams 🙌

The `Files` class has several overloaded methods that allow you to acquire binary as well as character streams from a file. They are easy to remember because the names and parameters of these methods follow a pattern. Their names start with `new` and their first parameter is always the `Path` to the file that you want to access. Depending on which overloaded method you are using, they have one or two more parameters - a `Charset`, in case you are trying to acquire a character stream, and a varargs array of `OpenOptions` that determine the mode in which the file is to be opened. The following are a few examples:

```
//newInputStream variants
```

```
InputStream is = Files.newInputStream(path);  
is = Files.newInputStream(path, StandardOpenOption.READ);
```

```
//newOutputStream variants
```

```
OutputStream os = Files.newOutputStream(path);  
os = Files.newOutputStream(path, StandardOpenOption.CREATE,  
StandardOpenOption.TRUNCATE_EXISTING);
```

```
//newBufferedReader variants
```

```
BufferedReader bfr = Files.newBufferedReader(path);  
bfr = Files.newBufferedReader(path, Charset.forName("UTF-8"));
```

```
//newBufferedWriter variants
```

```
BufferedWriter bfw = Files.newBufferedWriter(path); //uses default  
Charset  
bfw = Files.newBufferedWriter(path, Charset.forName("UTF-8"));
```

```
bfw = Files.newBufferedWriter(path, StandardOpenOption.WRITE);  
bfw = Files.newBufferedWriter(path, Charset.forName("UTF-8"),  
StandardOpenOption.CREATE);
```

Most of the parameters are self explanatory but the `OpenOption` parameter is a bit complicated. `OpenOption` is an interface and it does not define any value. The values are defined in the `StandardOpenOption` class, which implements `OpenOption`. All four methods mentioned above can be invoked without this argument and in that case, a default value is assumed. The defaults are different for each method and are as follows:

For `newInputStream`, the default is `READ`.

For `newOutputStream`, the default is the combination of `WRITE`, `CREATE`, and `TRUNCATE_EXISTING`.

The `newBufferedReader` method does not have this parameter because the file is opened only for reading.

For `newBufferedWriter`, the default is the combination of `WRITE`, `CREATE`, and `TRUNCATE_EXISTING`.

The combination of `WRITE`, `CREATE`, and `TRUNCATE_EXISTING` means that the file is opened for writing, it will be created if it does not already exist, and it will be truncated (i.e. all of its contents will be deleted) if it exists.

Furthermore, depending on whether you are opening a file for reading or writing, some `OptionOption` combinations may be illegal to use while acquiring the stream. For example, if you pass `StandardOpenOption.WRITE` to `newInputStream`, or `StandardOpenOption.READ` to `newOutputStream` an `IllegalArgumentException` will be thrown. You don't need to remember all combinations of valid and invalid values for the exam.

Reading and writing all data at once 🙌

If you feel that using streams to read or write small files is too much work, you will love the following methods:

Reading and writing bytes

```
byte[] allBytes = Files.readAllBytes(path);  
Files.write(path, allBytes); //OpenOptions may also be passed as the  
third argument
```

Reading and writing text

```
String data = Files.readString(path);  
List<String> allLines = Files.readAllLines(path);  
//Charset may also be passed as the second argument in both of the  
above  
  
Files.write(path, data);  
Files.write(path, allLines);  
//Charset and OpenOptions may also be passed as the second and the  
third arguments
```

Not only are the above methods convenient to use, they also close the underlying resource automatically after the reading/writing is complete without the need for wrapping them into a try-with-resource statement. The only catch is that if you try them on a large file, you may run out of memory.

Creating a Stream of lines 🙌

Since reading all the lines using the `readAllLines` method runs the risk of running out of memory, a better approach is to create a `java.util.stream.Stream` of lines out of the text file using the `lines` method:

```
try(Stream<String> streamOfLines = Files.lines(path) ){  
    //use the Stream API to process the data  
    streamOfLines.forEach(System.out::println);  
}
```

You will learn a lot more about the **Stream API** later but observe that I have used the try-with-resources statement above. That's because a call to the `Stream`'s `close()` method is required to release the underlying file lock.

Reading a directory 🙌

You can read the list of files in a directory using the `list` method:

```
try(Stream<Path> streamOfPaths = Files.list(path) ){
    streamOfPaths.forEach(System.out::println);
}
```

Walking a directory tree 🙌

If you want to recursively traverse through a directory tree in depth first manner, there are a couple of `walk` methods that return a lazily populated `Stream` of `Path`s:

```
Stream<Path> sp = Files.walk(Path.of("c:\\temp"),
    FileVisitOption.FOLLOW_LINKS);
List<Path> files = sp.filter(p → Files.isDirectory(p) &&
    p.getName(p.getNameCount()-1).toString().startsWith("a")).collect(Collectors.toList());
System.out.println(files);
```

The above code lists all the directories that lie under `c:\temp` directory at any depth whose names start with `a`. The above code may look scary if you have no idea about the Stream API. Come back to it after learning about Stream API later and you will see that it is actually quite straight forward and useful.

Checking file type and other file attributes 🙌

There are a few methods in `Files` that can be used to check the kind of file a `Path`

refers to, namely, `isRegularFile`, `isDirectory`, `isHidden`, `isReadable`, `isWritable`, and `isExecutable`. All of them take a `Path` (and a vararg of type `LinkOption` in some cases) and return a `boolean`. You may also check the existence of a file using the `exists` method.

Besides the above, there are `readAttributes`, `getAttribute` and `getFileAttributeView` methods to read file attributes. For example:

```
Path myFile = Path.of("c:\\temp\\test.txt");
BasicFileAttributes attr = Files.readAttributes(myFile,
BasicFileAttributes.class);
FileTime fileTime = attr.creationTime();
System.out.println("File creation time: " + fileTime);
```

Different file systems support different file attributes but all of them support a few basic attributes that are common to all file systems. These are represented by the `BasicFileAttributes` interface. Others such as `DosFileAttributes` and `PosixFileAttributes` extend `BasicFileAttributes`.

Although they are not explicitly mentioned in the exam, they are quite useful professionally.

Other utility methods in Files class 🙌

There are several methods to create, copy, move, and delete files as well as directories in the `Files` class. I am not listing them here because the exam doesn't quiz you on those although they may be used incidentally in code snippets. I suggest you to browse through the JavaDoc of the `Files` class to be aware of such methods.

19.3 Serialize and de-serialize Java objects 🙌

Java Serialization is readymade mechanism available in the standard Java library for capturing the state of an object in a form that can be saved and can be used later to restore an object to that same state. This mechanism was in vogue a decade ago before the advent of third party libraries that could do the same thing using cross-platform formats such as JSON and XML.

Practically, Java serialization has completely fallen out of favor and is almost never used nowadays because not only is it tightly coupled to Java, it is also a security risk. Unfortunately, the OCP certification exam still expects you to know about this mechanism in detail, most likely because countless libraries and frameworks depend directly or indirectly on this mechanism.

As usual, I will start with a simple example to give you an overview and then introduce the complications.

19.3.1 Serializing and deserializing an object 🙌

Imagine an `AppPreferences` class that is meant to store the location of a Window of a GUI application. It defines two instance fields `x` and `y` of type `int` and an instance field `title` of type `String`. Their values essentially constitute the state of an `AppPreferences` object. It is this state that we want to preserve in a file so that it can be used to initialize the view the next time the application is launched. The following code shows how we can save an `AppPreferences` object to a file and then recreate that object again by reading it from the file.

```
import java.io.*;
class AppPreferences implements Serializable{
    private int x;
    private int y;
    private String title;
    private transient String securityKey = "1234";
    //public getter/setter methods not shown
```

```

AppPreferences(int x, int y, String title){
    this.x = x; this.y = y; this.title = title;
    System.out.println("AP Created: "+x+" "+y+" "+title+" "+securityKey);
}
public String toString(){
    return "AP("+x+", "+y+", "+title+" "+securityKey+")";
}
}
public class SerializationTest {
    public static void main(String[] args) throws IOException,
    ClassNotFoundException {
        AppPreferences p = new AppPreferences(100, 100, "App");
        try(ObjectOutputStream oos = new ObjectOutputStream(new
        FileOutputStream("appprefs.ser"))) ){
            oos.writeObject(p);
        }
        try(ObjectInputStream ois = new ObjectInputStream(new
        FileInputStream("appprefs.ser"))) ){
            p = (AppPreferences)ois.readObject();
            System.out.println(p);
        }
    }
}

```

The above code produces the following output:

```

AP Created: 100 100 App 1234
AP(100, 100, App, null)

```

Let us look at various aspects of the above program one by one.

The Serializable interface

The first thing that you probably noticed about the above code is the `implements`

`Serializable` clause. To use Java's inbuilt serialization mechanism the class that we want to serialize must be "serializable", i.e., it must directly or indirectly implement the `java.io.Serializable` interface. Not only the class but its instances fields must also be serializable. This is a recursive requirement. So, if a class has an instance field of some other class, then that other class must also implement `Serializable` and that other class's instance fields' types must also do the same. It nests as deep as there are instance fields. If any object that is not serializable is found while serializing an object of a class then a `java.io.NotSerializableException`, which is an `IOException`, is thrown by the `writeObject` method.

Since static fields do not constitute the state of an object, they are never serialized. So, it doesn't matter whether a static field is serializable or not.

Fortunately, all primitives are serializable and most of the commonly used classes from the Java SDK such as `String` and wrapper classes (`Integer`, `Double`, etc.) implement `Serializable`. Arrays of primitives and serializable objects are serializable too. Thus, in the above code, the values of `x`, `y`, and `title` will be preserved in the serialized data. Note that `java.lang.Object` class does not implement `Serializable`. If it did, everything would be serializable!

Marker interface

The `Serializable` interface doesn't define any field or method. Its sole purpose is to indicate to the JVM that objects of this class are intended to be serialized and deserialized. Thus, it is just a "marker" interface.

Marking a class serializable is important from security perspective because an object's state may comprise confidential information and the developer may not want it shared outside of the JVM in which the application is running. Once an object is serialized and that serialized data goes out of the JVM, for example, once it is saved in a file, the JVM has no control over it. Anyone can tamper with it and in that case, recreating that object using tampered data may pose a serious security risk to the application. Implementing `Serializable` indicates to the JVM that we are okay with the risks involved in serializing and deserializing objects of this particular class.

In other words, Java doesn't allow its serialization mechanism to be used to serialize any random object unless the developer explicitly accepts the risks involved by implementing the `Serializable` interface. This is not to say that a serializable class is always a security risk. You may write additional custom code on top of Java's serialization mechanism to mitigate the risks associated with serialized data. These are described in detail in "Secure Coding Guidelines for Java SE" document published by Oracle. Although it is not required for the OCP exam, I strongly encourage you to go through it.

The `ObjectOutputStream` and `ObjectInputStream` classes 🙌

The `java.io.ObjectOutputStream` and `java.io.ObjectInputStream` classes contain the logic of serializing and deserializing objects respectively. `ObjectOutputStream`'s `writeObject` method takes a serializable object and converts it into a stream of bytes. This stream of bytes can be connected to any other output stream such as a `FileOutputStream`, as done in the above program. Similarly, `ObjectInputStream`'s `readObject` method reads a stream of bytes and creates the object back. This stream of bytes can be sourced from any other input stream such as a `FileInputStream`, as done in the above code. Just like other stream methods, these methods too throw `IOException`.

Observe in the above code that we are casting the object returned by `readObject` to `AppPreferences`. This is because the return type of `readObject` is `Object`. Since we know that the input file contains an `AppPreferences` object, we can safely cast the object returned by `readObject` to `AppPreferences`. Of course, if the input stream contains some other type of object that cannot be cast to `AppPreferences`, this cast will throw a `java.lang.ClassCastException`, which is an **unchecked exception**.

Another important point to note here is that although the above code writes and reads just one object to/from the output and input streams, there is no limitation to the number of objects you can write and read. For example, the following code writes all elements present in a list to the file and then reads them back:

```

var al = new ArrayList();
al.add(10.0); al.add(null); al.add("hello");
ObjectOutputStream oos = new ObjectOutputStream(new
FileOutputStream("data.ser"));
for (Object obj : al) {    oos.writeObject(obj);  }
oos.close();
try(ObjectInputStream ois = new ObjectInputStream(new
FileInputStream("data.ser"))) {
    Object obj = "";
    while (true) {
        obj = ois.readObject();
        System.out.println("Read " + obj);
    }
} catch (EOFException e) {
    System.out.println("All objects were read.");
}

```

Observe that the while condition in the above code is just `true`. We are not using `obj==null` as the while condition because a `null` value coming from `readObject` does not mean that the input stream has ended. It just means that a `null` reference was stored in the file. Once all of the contents of the file are read, a `java.io.EOFException` will be thrown and the while loop will break. This is confirmed by the following output:

```

Read 10.0
Read null
Read hello
All objects were read.

```

The `ClassNotFoundException` exception 🙌

The serialized data contains information about the class of the object that was serialized. Based on that information, `readObject` creates an object of that class. But for it to create an object of that class, it needs the class definition. Recall that class

definitions are contained in class files and for a class definition to be available to the `readObject` method, the corresponding class file (and all other class files that might be referred to by this class) must be present on the classpath. So, if `readObject` is not able to find any of the class definitions required to create the given object, it throws a `java.lang.ClassNotFoundException`. `ClassNotFoundException` is a **checked exception** but it does not extend from `IOException` and so it must be handled separately from `IOException` either using a `catch` clause or by declaring it in the `throws` clause, as done in the example above.

The `transient` keyword

If there are instance fields of a class that we don't want to be present in the serialized data, such as the `securityKey` field in the above code, we need to mark them `transient`. The serialization mechanism ignores all transient fields of a class while serialization. Since the serialized data doesn't contain any value for transient fields, such fields are initialized to their default values (i.e. `0`, `false`, or `null`) while deserializing the data. Observe in the above output that the `securityKey` field is `null` after deserialization.

Role of constructors and instance initializers during deserialization

You might have observed from the above output that the print statement from the constructor is not executed when the `AppPreferences` object is deserialized. That is because constructors and instance initializers of a serializable class are not executed when an object of that class is created through deserialization. The reason is simple. The purpose of constructors and instance initializers is to initialize the values of instance fields to sensible values as per the business logic of the class. However, these values may change during program execution. Thus, at the time of serialization, the values of the instance fields may not be the same as the values that were given to those fields by the constructor and/or instance initializers. Therefore, it doesn't make sense to execute constructors and instance initializers when we are trying to set the values of the fields of the object from the serialized data.

There are **two** exceptions to this rule, which you will read about shortly.

Serialization and inheritance 🙌

If a class implements `Serializable`, all its subclasses automatically become serializable. Although there are ways to prevent serialization of such a subclass (not important for the exam), there is no way to make the subclass "unimplement" the `Serializable` interface.

The more important case is when a super class of a serializable class is not serializable. The rule is that if a serializable class extends a non-serializable class then the no-args constructor (and instance initializers) of the non-serializable super class will be executed. This means that a non-serializable super class of a serializable class must have a no-args constructor. Check out the following code:

```
class Preferences{
    public String userName = "default";
    Preferences(String userName){
        this.userName = userName;
    }
}
class AppPreferences extends Preferences implements Serializable{
    AppPreferences(){
        super("Test User");
    }
}
public class SerializationTest { //not using try blocks for brevity
    public static void main(String[] args) throws Exception{
        AppPreferences p = new AppPreferences();
        ObjectOutputStream oos = new ObjectOutputStream(new
        FileOutputStream("c:\\temp\\appprefs.ser"));
```

```

    oos.writeObject(p); oos.close();
    ObjectInputStream ois = new ObjectInputStream(new
FileInputStream("c:\\temp\\appprefs.ser"));
    p = (AppPreferences)ois.readObject(); //InvalidClassException
    System.out.println(p.userName);
}
}

```

The above code will throw a `java.io.InvalidClassException` with the message `AppPreferences; no valid constructor from the call to the readObject method`. The message is mysterious because a constructor is indeed present in `AppPreferences` as well as in `Preferences`. However, since `Preferences` is not serializable, the JVM tries to invoke its no-args constructor, which it does not have. `InvalidClassException` is an unchecked exception.

This makes sense because the serialized data doesn't contain values for the fields of the non-serializable super class and those fields need to be initialized through a constructor and instance initializers. However, since the JVM does not have any arguments to pass, it can only invoke the no-args constructor.

Therefore, if we add a dummy no-args constructor to `Preferences`, i.e., `Preferences(){}`, `userName` will be initialized to `"default"` (and not to `"Test User"`) during deserialization.

19.3.2 Managing serialization during class evolution 🙌

We have been serializing and deserializing objects in the same test program and at the same time but the purpose of serializing objects is to be able to read them back again at any later date, which could be today or even years later. However, during this time a class may go through multiple changes such as bug fixes in the logic and addition and removal of instance fields. Even the JRE version being used on the production

machine may change between the time an object was serialized and the time it is being deserialized. Java serialization mechanism promises that if certain rules are followed, a serialized object can be recreated at any later date in spite of the changes described above.

Using `serialVersionUID`

The Java serialization mechanism creates a version number for a every serializable class based on the properties of that class. It includes this version number in the serialized stream. While deserializing the object, it tries to match the saved version number with the version number associated with the class definition. If the version numbers do not match then it assumes that the serialized data is incompatible with the class into which it is being deserialized and it throws a `java.io.InvalidClassException`.

The exact algorithm for version number generation is not important. The important thing is that this number may change even due to a slight change in the class definition or due to a change in Java version. For example, addition of an instance field to a class will cause it to generate a different version number even though you might not consider this an incompatible change and might want to create an object using the new class definition with whatever values are present in the old serialized data.

To prevent automatic version number generation, we need to define a **static final long** field named `serialVersionUID` in a serializable class. It can have any access modifier but **private** is recommended. It can be initialized to any value but conventionally, it is set to `1L`. If we do not want objects of a class to be created using serialized data that was created from an older version of the class, we can increment the version number. But as long as the value of this field stays the same, the serialization mechanism will try its best to recreate the object using the serialized data. Note that it does not a guarantee backward compatibility because there are certain changes that render a class incompatible such as moving a class up or down in the hierarchy or removing an instance field. You may go through the Java Object Serialization Specification to see the list of all such changes but it is not required for the OCP exam.

19.3.3 Customizing object serialization 🙌

Usually, the readymade serialization logic contained in `ObjectInputStream` and `ObjectOutputStream` classes is sufficient to read and write serialized objects. In the rare case that you want to tweak this logic there are a couple of approaches.

Implement `writeObject`, `readObject`, and, `readObjectNoData` methods

In this approach, your class must still implement `Serializable` but besides that, it may also provide implementation for the following three methods:

1. `private void writeObject(java.io.ObjectOutputStream out) throws IOException`
2. `private void readObject(java.io.ObjectInputStream in) throws IOException, ClassNotFoundException`
3. `private void readObjectNoData() throws ObjectStreamException`

For example, the following class customizes deserialization of its objects by implementing the `readObject` method:

```
class Vehicle implements Serializable{
    private String brand;
    private transient int rating;
    public Vehicle(String brand, int rating){
        this.brand = brand; this.rating = rating;
    }
    private void readObject(ObjectInputStream ois) throws Exception{
        //must be private
        ois.defaultReadObject(); //this call is required
        rating = 100;
    }
}
```

The above `readObject` method calls `ois.defaultReadObject()` to ensure that the fields of the `Vehicle` object are set using the values available in the serialized data and then sets the value for the `rating` field on its own using a hardcoded value.

Although the OCP 21 exam does not go into the implementation details of `readObject` / `writeObject` methods, it does expect you to know that their access modifier must be **private**. So, for example, if a class has a `readObject` method that is public, it won't be invoked during deserialization of an object of that class and deserialization will occur normally.

Implement Externalizable

The second approach is to implement the `java.io.Externalizable` interface instead of implementing the `Serializable` interface. This interface contains two abstract methods that the class must implement:

1. `void readExternal(ObjectInput in)`
2. `void writeExternal(ObjectOutput out)`

I am not going into the details of these approaches because neither are they required for the exam nor are they used professionally very often. It is good to be aware of these approaches for technical interviews though. You may go through "Java Object Serialization Specification" for more details if you have time.

19.3.4 Serializing Records and Enums

Serializing records

A record too can be serialized if it implements `Serializable`. However, there are a couple of fundamental differences between serialization of regular classes and that of records. These are as follows:

1. Records are not allowed to customize their serialization and deserialization, which means you can't provide `writeObject` and `readObject` methods for a record. Since records are immutable, the state of a record always remains the

1. same once it is created and is completely determined by the value of its components. Therefore, there is never a need to customize the serialization and deserialization of a record.
2. The canonical constructor of a record is invoked even when it is being created by deserializing the record data. This is because Java language designers preferred to have exactly one way to create record instances to avoid security risks associated with bypassing constructors while initializing instance fields using reflection.

Serializing enums

As you know, all enums implicitly extend `java.lang.Enum`, which implements `Serializable`. Therefore, all enums are serializable. However, the protocol for serialization of enums is also quite different from regular classes. You can read about it in the Java Object Serialization Specification but it is not required for the OCP exam.

19.3.5 Quiz

Q. What fixes are required for the following code to compile and run without any exception:

```
class Document{
    byte[] data;
    Document(byte[] data){
        this.data = data;
    }
}
class MimeDocument extends Document implements Serializable{
    String type;
    MimeDocument(byte[] data, String type){
        super(data);
    }
}
```

```

        this.data = data; this.type = type;
    }
}

public class SerializationTest {
    public static void main(String[] args) {
        Document doc = new MimeTypeDocument(new byte[100], "bmp");
        ObjectOutputStream oos = new ObjectOutputStream(new
FileOutputStream("c:\\temp\\doc.ser"));
        oos.writeObject(doc); oos.close();

        ObjectInputStream ois = new ObjectInputStream(new
FileInputStream("c:\\temp\\doc.ser"));
        doc = (MimeTypeDocument)ois.readObject();
        System.out.println(doc.type+" "+doc.data);
    }
}

```

Select **3** correct options.

- A. Add `implements Serializable` to `Document`
- B. Add a no-args constructor to `Document`
- C. Either add `implements Serializable` or add a no-args constructor to `Document`
- D. Add `throws IOException, ClassCastException` clause to the `main` method
- E. Add `throws IOException, ClassNotFoundException` clause to the `main` method
- F. Change the type of `doc` variable from `Document` to `MimeTypeDocument`
- G. Add `private static final long serialVersionUID=1L;` to `MimeTypeDocument`

Correct answer is C, E, F.

If a class does not implement `Serializable`, its no-args constructor is invoked while deserialization of its serializable subclass. Therefore, **option C** is correct.

The `readObject` method may throw an `IOException` as well as a `ClassNotFoundException`, both of which are checked exceptions and since neither of them is a subclass of the other, both of them must be mentioned in the throws clause. Therefore, **option E** is correct. Note that the cast may throw a `ClassCastException` but it is an unchecked exception.

Since the `type` field is in `MimeDocument` class, `doc.type` will not compile unless the type of `doc` is `MimeDocument`. Therefore, **option F** is correct.

Although defining `serialVersionUID` in a class that implements `Serializable` is recommended as a good practice, it not necessary.

19.4 Writing to and reading from console

Conceptually, a console is the dashboard that allows a user to control various aspects of the system through various knobs and levers. In modern computers, a console is a text based window from where a user can interact with the Operating System by executing shell commands. On a Windows machine, it is the DOS prompt or the command prompt and on a Linux machine it is the command shell such as bash. Advancements in GUIs has made consoles nearly obsolete for end users but for programmers, consoles are indispensable during development and testing of programs primarily because they allow programmers to interact with the program without spending any time in developing a custom GUI. They provide an easy way for a program to output information to the programmer and also to accept input from the programmer while the program is executing.

A console is not available when you launch a program by double clicking an executable file (or by double clicking a file whose type is associated with an executable) from the file explorer. In such a case, the only way for the program to interact with the user is to create custom GUI windows using a windowing toolkit or an API. But when you execute a program from the command line, the program automatically gets access to that command window. It can write textual data to it and can collect user input through the same with very little effort. The same is true for a Java program as well. In this section, you will learn how to make use of the console in

a Java program.

19.4.1 Using `System.out` and `System.in` 🙌

Writing to the console using `System.out`

The easiest way to output text to the console is to use `java.lang.System` class's **public static** variable named `out`. Indeed, we have been using it since our first Java program! The `out` variable is of type `java.io.PrintStream` and the `PrintStream` object pointed to by this variable is connected to the console. That is why whatever we print using its `print` or `println` methods, it shows up on the console.

A `PrintStream` is a `FilterOutputStream` and is used to "print" representations of various data values conveniently. In addition to the overloaded `write` methods, which accept a `byte[]`, it has several overloaded `print` and `println` methods that accept various primitive data types as well as `String` and `Object`. Its `printf` method is particularly useful for outputting formatted text. You should check out the JavaDoc description of this class for details but let me mention a couple of important point about this class here anyway.

First, even though a `PrintStream` is able to print all sorts of objects including strings, it is technically a binary stream, which means, it writes binary data to its destination (which is the console in case of `System.out`). So, whenever you use its `printXXX` methods, it converts the input into bytes using the given encoding or charset, or the default charset if not specified. **Second**, unlike other I/O stream classes, methods of `PrintStream` do not throw any checked exception, which is why a Java beginner's "Hello World" program can be written without worrying about exception handling. To check if any error occurred while writing any data to the underlying stream, its `checkError()` method can be called.

Besides `System.out`, there is `System.err` as well, which is also of type `PrintStream`. This stream is the "standard error stream" and is used by the JVM to print exception stack traces

(the output of `Throwable`'s `printStackTrace()` method goes to this stream). Usually, this stream is also connected to the console, which is why you see exception stack traces printed on the console along with other data sent to `System.out`. If you use an IDE to execute a Java program, you may see data sent to the error stream printed in a different color or on a different window altogether.

Getting input from the user using `System.in`

`System.in` refers to an `InputStream` object that is usually connected to keyboard input of the machine. It can be used as follows to accept input from the user:

```
public class TestClass{
    public static void main(String[] args) throws IOException{
        int value = System.in.read();
        System.out.println("You entered "+ value);
        System.out.println("You entered "+(char) value);
    } }
```

When you run the above program, it waits for you to enter some input until you hit the enter key. For example, if you press 1 and hit enter, it will print:

```
You entered 49
You entered 1
```

The following points are worth noting about the above code:

1. Since methods of `InputStream` may throw `IOException`, a call to `read` requires code for handling that exception.
2. Since the `InputStream` is sourced from the keyboard, it returns a byte stream representing the characters that are pressed on the keyboard. So, when you press `1`, the keyboard actually sends a byte containing the ASCII code (assuming that the keyboard is an ASCII keyboard) for `1`, which is `49`.
3. The keys pressed by the user do not flow through to the stream until the user

3. presses the enter key, which is why the program waits till you hit the enter key before printing anything.
4. The above program reads only the first byte from the stream. If you want to read all the keys pressed by the user, you may use a while loop like this:

```
while( true ){  
    int value = System.in.read();  
    System.out.println("You entered "+ value);  
}
```

You may also read a complete line from the console by wrapping `System.in` into a `BufferedReader` :

```
BufferedReader bfr = new BufferedReader(new  
InputStreamReader(System.in));  
String line = null;  
while( !(line = bfr.readLine().trim()).equals("")){  
    System.out.println("You entered "+ line);  
}  
System.out.println("End");
```

The above code will keep accepting and printing lines of text from the user until they input an empty line. Recall that a line of text is terminated by any one of a line feed (`'\n'`), a carriage return (`'\r'`), a carriage return followed immediately by a line feed, or by the end of the file. If you are wondering why `System.out` refers to a `PrintStream` instead of some `Writer` such as `PrintWriter` if all we use it for is to print text to the console, then you should know that the target of a program's output may not always be a console. Its output could potentially be piped to another program that expects binary data. Therefore, it is quite important for the output stream of a program to be able to carry binary data as well. The `PrintStream` class supports writing binary as well as text. A program can write textual data using `PrintStream`'s `print` /

`println` methods and binary data using any of the variants of its `write(byte[] b)` methods.

The same reasoning applies to the input stream of a program. A program's input source may not always be a console and so, the input of a must be capable of delivering binary data to the program. If a program knows that it is always going to get data from the console as its input, it can either wrap the input stream into a Reader (as shown above) or use the `Console` class, which you will see next.

19.4.2 Using `java.io.Console` 📌

The `Console` class provides a higher level abstraction for the console associated with the Java program. As you will observe from the examples below, it is a bit more convenient to use and also has a few advantages over using `System.in` and `System.out` directly.

Acquiring the `Console` object

The `Console` class does not have any public constructor and so you cannot create a `Console` object using `new`. There is only one `Console` object for the entire application and you must acquire it by calling `System.console()`. The important thing to know about this method is that it returns `null` if no console is associated with the JVM. This may happen, for example, when an application is launched in the background using **javaw**. Unfortunately, console is not available even when you run your program in IDEs such as Netbeans and Eclipse because these IDEs too execute the program using a background process. But they do redirect the `System.in` and `System.out` to a window in the IDE, so you can still use the streams directly for printing and accepting user input.

Writing to console using `Console`

The `Console` class has the following two methods to print text on the console:

1. `Console format(String format, Object... args)` - Writes a formatted string to this console's output stream using the specified format string and arguments. For example, the following code prints, `Hi Bob, how are you? :`

```
Console c = System.console();  
if(c≠null){ // this check is important  
    c.format("Hi %s, how are you?", "Bob");  
}
```

Complete details of the format specifiers are not required for the exam but knowing about a few commonly used specifiers such as `%s`, `%d`, `%f`, and `%t` can be helpful while writing professional code.

2. `Console printf(String format, Object... args)` - This is a convenience method that works exactly the same way as the `format` method.

Observe that the above two methods return a reference to a `Console`. It is a reference to the same `Console` object on which the methods are invoked. The purpose of the return value is to enable chaining multiple method calls in the same statement. For example, `c.format("Hi!").format("Good Morning");`

If you prefer the methods provided by

`PrintWriter`, there is also a `writer()` method in `Console`, which returns a `PrintWriter` object that writes to the associated console.

Reading from console using `Console`

Accepting user input is where the `Console` class really shines. It has the following four methods for this purpose:

1. `String readLine()` - Reads a single line of text from the console.
2. `String readLine(String fmt, Object... args)` - Provides a formatted prompt, then reads a single line of text from the console.
3. `char[] readPassword()` - Reads a password or passphrase from the console with echoing disabled
4. `char[] readPassword(String fmt, Object... args)` - Provides a formatted prompt, then reads a password or passphrase from the console with echoing disabled.

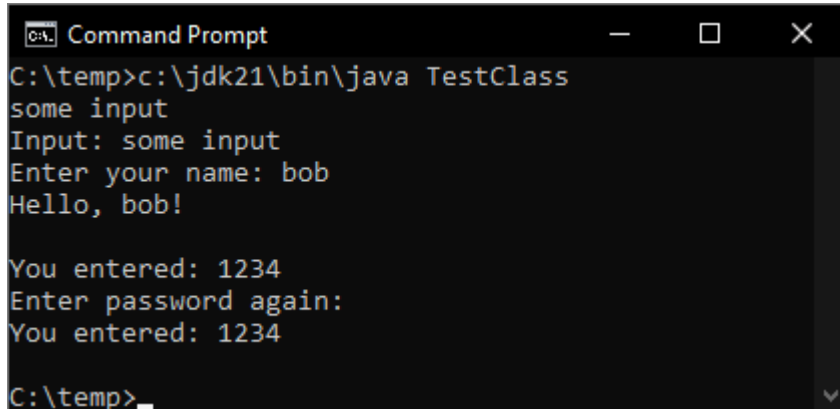
The above methods return a complete line of input (without the line terminating characters at the end). The following code illustrates how the above methods work:

```
public static void main(String[] args) throws IOException{
    Console c = System.console();
    if(c == null){ System.out.println("no console"); return; }

    String input = c.readLine();
    c.format("Input: %s \n", input);
    String name = c.readLine("Enter your name: ");
    c.format("Hello, %s!\n", name);
    c.format("Enter password :");
    char[] pwd1 = c.readPassword();
    c.format("You entered: %s\n", new String(pwd1)); //for illustration only
    char[] pwd2 = c.readPassword("Enter password again: ");
    c.format("You entered: %s\n", new String(pwd2)); //for illustration only
    if(!Arrays.equals(pwd1, pwd2)){ System.out.println("Pwds
```

```
don't match"); }  
    //else do something with the pwd  
    Arrays.fill(pwd1, ' '); Arrays.fill(pwd2, ' '); //recommended  
}
```

The output of the above program is shown in the screenshot below:



```
Command Prompt  
C:\temp>c:\jdk21\bin\java TestClass  
some input  
Input: some input  
Enter your name: bob  
Hello, bob!  
  
You entered: 1234  
Enter password again:  
You entered: 1234  
C:\temp>
```

The `readLine` method that takes no argument is unremarkable. It just returns the complete line of input entered by the user. I entered `some input` and the same is printed by the format method. The `readLine` method with a format string argument provides a textual hint to the user about what they need to enter. Observe that in both the methods, the keys pressed by the user are **echoed** to the console. That is why you see `some input` and `bob` written twice in the above screenshot.

The `readPassword` methods works similarly but they do not echo the keys pressed by the user to the console. This is useful when you want the user to enter sensitive information that should not be displayed on the console to prevent someone watching over the console from seeing it. The data returned by the `readPassword` methods is **not encrypted** but note that these methods return a `char[]` instead of a `String` so that you can clear the char array (such as by using `Arrays.fill` method) after making use of the password. This ensures that the password typed by the user will not be revealed even by dumping program memory. It would not have been possible to do so if they returned a `String` because strings are immutable.

19.5 Exercise

1. Write a method `void takeBackup(String completeFilePath)` that does the following:
 - Finds out the last modified time of the give file
 - Creates a new file in the same directory as the original file but with the modified time appended to its name.
 - Copies the original file to the new file.

Hint: Use `readAttributes`, `resolveSibling`, and `copy` methods.